

Project APEX Ten-Project Instructor Ladder

Progressive builds that culminate in the production simulation engine

OpenAI

August 2026

Table of Contents

1 How the project ladder works

Each project is larger than a lab and smaller than the final system. Start from an empty file. Use the supplied implementation only after your acceptance tests run against your own attempt. A project is complete when you can explain its contracts, reproduce its results, inject its listed failure and compare at least one alternative.

Use one branch or directory per project. At the end, write an Architecture Decision Record with: context, options, evidence, decision, consequences and revisit condition.

2 Project 1: One-Dimensional Car Integrator

2.1 Mission

Create an explicit state/action transition and a deterministic rollout.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

2.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

2.3 Acceptance criteria

Position and velocity arrays match an analytical constant-acceleration case.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

2.4 Deliberate failure

Wrong time-step use and update order.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

2.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

2.6 Transfer into APEX

The simulation interface used by every later model.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

2.7 Repository

projects/01_car_integrator/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

3 Project 2: Telemetry Quality Laboratory

3.1 Mission

Define a typed telemetry contract and reject corrupt units, ranges, timestamps and axes.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

3.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

3.3 Acceptance criteria

Every injected defect fails at the intended boundary with a useful message.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

3.4 Deliberate failure

Silent coercion and semantic swaps.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

3.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

3.6 Transfer into APEX

Canonical contract and ingestion quality gate.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

3.7 Repository

projects/02_telemetry_quality/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

4 Project 3: Lap Physics Sandbox

4.1 Mission

Add track progress, curvature demand, grip, tyre degradation and weather to a causal toy environment.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

4.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

4.3 Acceptance criteria

Changing each scenario variable produces a directionally justified trajectory change.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

4.4 Deliberate failure

Impossible controls and discontinuous progress.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

4.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

4.6 Transfer into APEX

Offline data source and trusted evaluator for planner tests.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

4.7 Repository

projects/03_lap_physics/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

5 Project 4: Nonlinear Transition Model

5.1 Mission

Train an MLP to predict next state and compare with persistence and linear controls.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

5.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

5.3 Acceptance criteria

Model improves matched test transitions and residual plots explain the gain.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

5.4 Deliberate failure

Target leakage and one-step-only conclusions.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

5.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

5.6 Transfer into APEX

First learned dynamics component.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

5.7 Repository

projects/04_mlp_transition/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

6 Project 5: Track Segment Encoder

6.1 Mission

Represent cyclic progress and local geometry without discontinuity at start/finish.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

6.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

6.3 Acceptance criteria

Nearby physical positions have nearby representations across the wrap boundary.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

6.4 Deliberate failure

Treating progress as an ordinary unbounded scalar.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

6.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

6.6 Transfer into APEX

Geometry/context representation for unseen-track transfer.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

6.7 Repository

projects/05_track_encoder/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

7 Project 6: GRU Telemetry Forecaster

7.1 Mission

Encode history and generate an autoregressive future under supplied controls.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

7.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

7.3 Acceptance criteria

Free-running horizon curves beat controls within a defined trusted range.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

7.4 Deliberate failure

Hidden-state reuse and teacher forcing at test.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

7.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

7.6 Transfer into APEX

Primary deterministic APEX V1 model.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

7.7 Repository

projects/06_gru_forecaster/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

8 Project 7: Selective SSM Memory Laboratory

8.1 Mission

Build stable input-dependent retention and compare with GRU on long/rare dependencies.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

8.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

8.3 Acceptance criteria

Selection provides measurable memory or latency value under matched budget.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

8.4 Deliberate failure

Exploding state and architecture-name overclaiming.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

8.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

8.6 Transfer into APEX

SSM challenger and Mamba preparation.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

8.7 Repository

projects/07_selective_ssm/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

9 Project 8: RSSM Imagination Laboratory

9.1 Mission

Separate posterior evidence from prior imagination and train stochastic latent dynamics.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

9.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

9.3 Acceptance criteria

Prior rollouts are useful and uncertainty diagnostics are calibrated on a stochastic task.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

9.4 Deliberate failure

Posterior leakage and KL collapse.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

9.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

9.6 Transfer into APEX

Dreamer-style belief model challenger.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

9.7 Repository

projects/08_rssm_imagination/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

10 Project 9: Latent MPC Planner

10.1 Mission

Use CEM to optimize bounded action sequences through a learned model.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

10.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

10.3 Acceptance criteria

Selected actions improve trusted-environment outcome without OOD exploitation.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

10.4 Deliberate failure

Reward loopholes and variance collapse.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

10.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

10.6 Transfer into APEX

Planning layer after dynamics certification.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

10.7 Repository

projects/09_latent_mpc/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.

11 Project 10: Project APEX Production Engine

11.1 Mission

Integrate ingestion, contracts, windows, models, evaluation, pipeline, registry, API and UI.

Do not begin with a framework. First draw the state, inputs, outputs and one transition. Write the acceptance test before the implementation.

11.2 Build sequence

Create a minimal executable entry point.

Define input/output types, axes, units and invariants.

Implement the smallest correct path.

Add one deterministic example with a hand-computed answer.

Add structured diagnostics or plots that expose internal state.

Introduce the deliberate failure listed below.

Repair it and add a regression test.

Compare one simpler and one more complex implementation.

Write the decision record.

Connect the project artifact to the next project without copying hidden assumptions.

11.3 Acceptance criteria

A fresh run is reproducible, tested, traceable and usable through the interface.

The criterion must be measurable. “It runs” is never sufficient. Save a report with configuration, seed, environment, output paths and limitations.

11.4 Deliberate failure

Artifact mismatch, UI false precision and source coupling.

Before opening any debugging note, predict the symptom, the earliest failed contract and the misleading downstream symptom. Preserve the failing artifact so the repair can be proven.

11.5 Implementation-choice review

Answer:

What is the simplest implementation that can satisfy the acceptance test?

What evidence would justify a more complex one?

Which assumption is encoded in the data rather than the model?

Which error is numerical, statistical, causal or operational?

What result would cause you to abandon this design?

11.6 Transfer into APEX

The capstone and base for F1 25/research expansion.

Locate the corresponding production module and explain what extra responsibilities appear there: configuration, batching, validation, checkpointing, artifacts, API compatibility, monitoring or UI provenance.

11.7 Repository

projects/10_project_apex/

Run the project, then rename or hide the solution and recreate its core from the README and tests. The second implementation is the real mastery check.